CS 4510 Automata and Complexity Due: 12:30pm April 3

Exam 2: Context Free Languages

Released: 12:30pm April 2

This exam was released at 12:30 EST Thursday, April 2 and is due at **12:30 pm EST on Friday, April 3**. There are no late submissions; leave yourself enough time to upload your work.

You should submit a typeset or *neatly* written pdf on Gradescope. The grading TA should not have to struggle to read what you've written.

This exam is open book and open notes. However, you may NOT use the internet to look up answers, and you may NOT discuss the exam with your fellow students or anyone else besides the TA's and the professor before the exam is due.

If you have questions about the exam, there will be a pinned thread on Piazza monitored by the TA's.

0. **Honor Code** 1 point

   I commit to uphold the ideals of honor and integrity by refusing to betray the trust bestowed upon me as a member of the Georgia Tech community.

   Signature: Aamod Varma

1. **Short Answer** *Section B only* (6 points)

   (a) (2 points) Explain why the following grammar is ambiguous. Your explanation should be specific to this grammar and should include justification.

$$S \to aA \mid Bb \mid \varepsilon$$
$$A \to Ab \mid \varepsilon$$
$$B \to aB \mid \varepsilon$$

   Then give an unambiguous grammar that generates the same language.

   Answer:

   (b) (2 points) Let $L = \{w \in \{a, b, c, d\}^* \mid w$ has an equal number of a's and b's, and an equal number of c's and d's $\}$. You want to use the pumping lemma to show L is not context-free. You have already assumed L is context-free, and set p to be the pumping length for L. Which, if any, of the following strings can you choose for the string s that is pumped and leads to a contradiction? (Circle all that apply. No explanation needed.)

   - $abcd$
   - $a^p b^p c^p d^p$
   - $(abcd)^p$
   - $a^{p^2} c b^{p^2} d$

Answer:

(c) (2 points) Let G be a context free grammar with one nonterminal and n rules. If G generates a finite number of words, what is the maximum possible number of words generated by G? Justify your answer.

Answer:

2. **Context-Free Grammars** *Both Sections* (6 points)
Give a CFG for each of the following languages:

(a) (3 points) $L_1 = \{a^i b^j c^k d^l \mid i + j = k + l, i, j, k, l \geq 0 \text{ are integers}\}$

Answer:We have,

$$S \to A \mid \varepsilon$$
$$A \to aAd \mid aCc \mid bBc \mid bDd \mid \varepsilon$$
$$C \to aCc \mid bBc \mid \varepsilon$$
$$D \to bDd \mid bBc \mid \varepsilon$$
$$B \to bBc \mid \varepsilon$$

(b) (3 points) $L_2 = \{ww^R \mid w \text{ is a binary string and contains exactly one } 1\}$

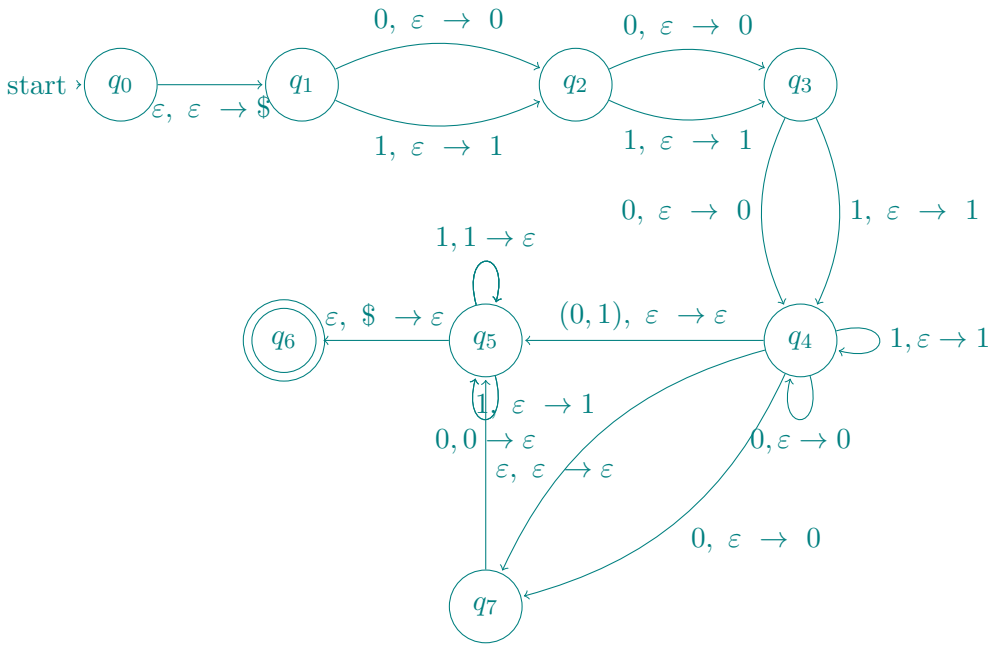
Answer:We have,

$$S \to 0S0 \mid 1B1$$
$$B \to 0B0 \mid \varepsilon$$

3. **Pushdown Automata** *Both Sections* (6 points)
Draw (by hand or with some visualization software) a PDA for each of the following languages:

(a) (3 points) $L_3 = \{x \mid x \text{ is a binary palindrome of length 7 or greater}\}$

Answer:

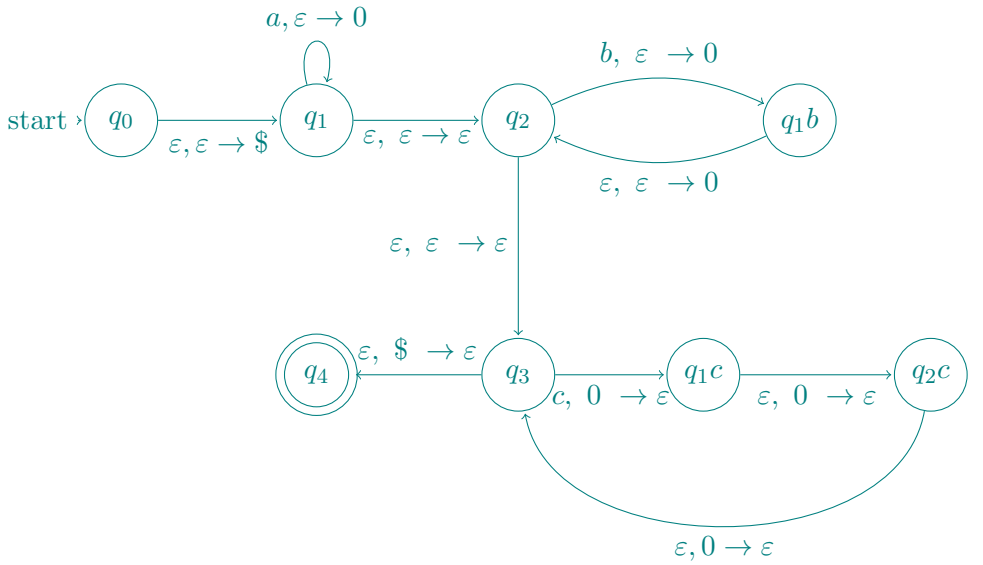


In this PDA we first push \$ into the stack and move to q_1. At q_1 consume the input and push it to the stack, we do this thrice to get to q_4. First time we reach q_4 we have three elements in the stack meaning we consumed three inputs. Now note that we can consume more inputs by the self transition from q_4 to itself. At this point we have the choice to take a guess. We can guess the middle element (in the case of odd palindrome) or we can guess the middle left character of the center (in the case of even palindrome). In the former case we guess the middle and do $(0, 1), \varepsilon \to \varepsilon$ which for any input will just ε transition to q_5, and in the latter we once again push what is presumably the last element in the first half and move to q_7. Now q_7 will ε transition without changing the stack to q_5. Now lastly q_5 will compare the next input to the top of the stack and pop it, lastly if the stack has \$ in the top we move to q_6 which is the accept state.

Note that the reason we need an addition state q_7 to handle the even case is that for even length palindrome we have either length 6 or 8, going directly to q_5 will allow the first half of our palindrome to be of length 3 implying we accept palindromes of length 6 which we do not want, hence we move to state q_7 to ensure the first half is at least length 4, for odd length as we consume the input while moving to q_5 we ensure that we consumed at least 4 elements and we have 3 elements in our stack left to pop, ensuring the minimum string length is 7.

(b) (3 points) $L_4 = \{a^i b^j c^k \mid i, j, k \in \mathbb{Z}^{\geq 0}, i + 2j = 3k\}$

Answer:



So here the idea is that everytime we see a we push 0 onto the stack once, every time we see b we push 0 onto the stack twice, and everytime we see c we pop the top of the stack thrice. This effectively ensures that $i + 2j = 3k$ and lastly if we see \$ we move to the accept state.

4. **Pumping lemma** *Both Sections* (6 points)

   Prove that $L_5 = \{w | a, b, c \in \Sigma$ w has the same number of each character and all the same characters are not contiguous (if there is more than one, there must be at least one set of characters that is separated by another character)$\}$ is not context-free. This means aabbcc, ccbbaa are not in the language for example, but abbacc or cababc is. abc can be in the language.

   Answer:Assume that the language is context free, with pumping length p. Now consider the string $w = ca^{p+1}b^{p+1}c^p$. Clearly $|w| > p$ and we have $p + 1$ number of a, b and c with non-contiguous number of $c's$. Now according to the pumping lemme we can write $w = uvxyz$ such that $|vxy| \leq p$ with $|vy| > 0$ for which $uv^ixy^iz \in L$ for any natural i. Now note that in our case we have $w = ca^{p+1}b^{p+1}c^p = uvxyz$. Now consider the choices for $|vxy|$. We have couple cases,

   (a) vxy lies entirely in one string. So in this case either it lies entirely in a, b or c. WLOG take it lies in a. So $vxy = a^j$ for some $j \leq p$. Clearly by pumping this with $i = 0$ in v^ixy^i we can reduce the number of $a's$ (we know this because $|vy| > 0$ so there is at least one a in that set, so by setting $i = 0$ we necessarily remove at least one a). But note that this means that our string is no longer in L as we've removed a singular a but not b or c hence having each character not be equal in number.

   (b) vxy does not lie entirely in one character. Note that in this case we have three possible choices.

i. vxy is a left substring of ca^{p+1} i.e $vxy = ca^k$ for $k \leq p-1$. In this case because we're assuming vxy does not lie entirely in one character this means that v contains the first character i.e c. Now at this point regardless of the exact arrangement, if v contains the c then we can always pump it using v^0xy^0 to get rid of the c. So this means we can remove one c or at least one a (or both). But note that we're modifying the number of $c's$ and $a's$ but leaving the $b's$ unchanged. This breaks the membership into the language.

ii. vxy is a substring of $a^{p+1}b^{p+1}$ including both characters, i.e it's of the form $vxy = a^ib^{j-i}$ for some $j \leq p$ and i a natural from 1 to $j-1$. Now note that in this case we have v contains an a and y contains a b. In both cases if we pump the string to zero i.e v^0xy^0 we reduce the number of a and b by at least one. However, doing so makes to number of characters unequal (as c stays the same) and hence out of our language.

iii. vxy is a substring of $b^{p+1}c^p$ that includes both b and c. So it is of the form $vxy = b^ic^{j-i}$ for some $j \leq p-1$ and i from 1 to $j-1$. If this is the case then we have v has at least one b and y has at least some c. If we pump using v^0xy^0 then we reduce the number of b and c by at least one each. But this kicks us out of the language as the number of $a's$ are note preserved.

Hence if vxy does not lie entirely in one character, we can pump the string outside the language for any choice of vxy.

As for any case of decomposition of w we showed that we can pump the string outside of the language, this means that our assumption is wrong and that the language is not context-free.

5. **k-depth PDA** *Section X only* (6 points) For any positive integer k, let a k-depth PDA be a PDA that can manipulate the characters at a specific index of the stack, rather than just the top; in particular, it can manipulate only up to the k'th character from the top. Thus the transition function contains elements of the form $(p, x, a, i) \to (q, b)$, where i is a positive integer less than or equal to k. The given transition signifies "In state p, when reading an x from the input and the stack is at least i characters tall with an a in the i'th position from the top, switch to state q, remove a from the stack directly and put b in its place." As before, x, a, and b may be ε.

   Notice then that a 1-Depth PDA is just a PDA. Prove that for any k-depth PDA P there is a regular PDA P' such that $L(P) = L(P')$.

   As a hint, first consider the case of showing there exists a PDA for every 2-Depth PDA.

   Answer:Firstly we show there exists a PDA for every 2-Depth PDA. Firstly the key idea is that in order to reach the second element of the stack and change it, we would need to remember what the first element of the stack is, so this means we need states that encode this the information about the first element of the stack. Note that in addition to this we also need to encode information about what element to replace, and what to replace it with and what state to go next. So essentially for every transition $(p, x, a, 2) \to (q, b)$ (note for $i = 1$ the case is trivial as it's just the

normal PDA transition), for each possible top of the stack say α we will add the state (α, a, b, q). So now we have the transition for each α that $(p, x, \alpha) \to ((\alpha, a, b, q), \varepsilon)$, note that what this does is seeing input x and top of the stack α it moves to the state (α, a, b, q) (note this 4 tuple is a state itself) while popping the top of the stack α. Now at this state, we will do the following. Denote $s = (\alpha, a, b, q)$ We will have the transition $(s, \varepsilon, a) \to (s_d, b)$ (this replaces a with b i.e the second element of the stack a with b and goes to the new state s_1) and then we'll add $(s_1, \varepsilon, \varepsilon) \to (q, \alpha)$ (this pushes α back to the top and goes to the state q). Hence effectively, we have replaced the second element of the stack from a to b while preserving the first element α and moving to the state q.

Now to generalize this to some k-Depth PDA the idea is essentially the same, however unlike in the 2-Depth PDA we need to encoder information about the first $i-1$ states. So consider a transition in the k-Depth PDA say $(p, x, a, i) \to (q, b)$. We will simulate this in a normal PDA. First for each possible value of i, note that we need to save the first $i-1$ states. If $|\Gamma| = k$ then this means k^{i-1} possible different combinations of characters filing the first $i-1$ positions. So for each possible combination for the first $i-1$ in our stack say $a_1 \dots a_{i-1}$ we need a state $(a_1, \dots, a_{i-1}, a, b, q)$ (note a is the $i'th$ element of the stack, b is what to replace it with and q is the final state to go to). Now we need to allow to transition from p to this state. In order to do that we need intermediate states. Let $(a_1, \dots, a_{i-1}, a, b, q)$ be named state s. So now we add transitions $(p, x, a_1) \to (s_1, \varepsilon)$, $(s_1, \varepsilon, a_2) \to (s_2, \varepsilon) \dots (s_{i-2}, \varepsilon, a_{i-1}) \to (s, \varepsilon)$. Now we've reached s and at this point we've popped the first $i-1$ elements of the stack. Now we add the transition to replace the i'th element with b so we add $(s, \varepsilon, a) \to (s'_1, b)$, now we need to go to state q, we need intermediate states for this. So we add $(s'_1, \varepsilon, \varepsilon) \to (s'_2, a_{i-1})$, $(s'_2, \varepsilon, \varepsilon) \to (s'_3, a_{i-2}), \dots, (s'_{i-1}, \varepsilon, \varepsilon) \to (q, a_1)$. So essentially we've added back all the states we popped in the start and hence we're left at state q with the $i'th$ element from a to b and all other elements untouched. Hence, simulating the k-Depth PDA with a normal PDA